

AI 情侣法庭微信小游戏端到端技术方案

Love Court · 云开发架构 / 数据存储 / API 分发 / AI 调用方案

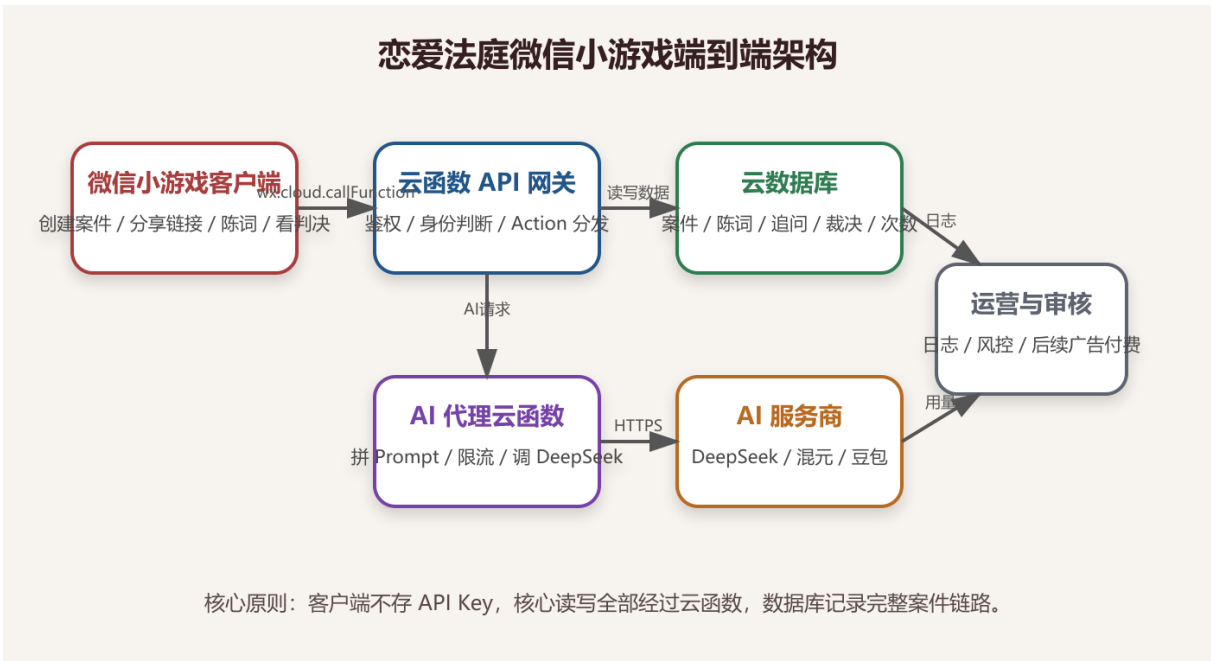
一句话结论

短期采用微信小游戏 + 微信云开发 + 云数据库 + 云函数 + DeepSeek，不自建服务器。客户端只负责展示和交互，所有核心数据读写与 AI 调用都经过云函数，避免 API Key 暴露，并为后续广告、次数和支付扩展预留空间。

1. 产品与技术目标

- 手机可玩，用户可在微信内直接创建案件、分享给对方并查看结果。
- 双方联机，原告创建案件后分享链接，被告进入后自动绑定身份。
- AI Key 安全，所有 AI 调用由云函数代理，前端不保存密钥。
- 数据可存档，案件、陈词、追问、裁决和调用记录都进入云数据库。
- 成本可控，MVP 不买服务器，先依赖微信云开发能力。

2. 端到端总体架构



架构说明：小游戏客户端通过 wx.cloud.callFunction 调用云函数。云函数先校验 openid、案件身份、邀请

码和调用次数，再读写云数据库；需要 AI 时，由 AI 代理函数拼接 Prompt 并调用 DeepSeek。返回结果写入 verdicts 后再返回客户端展示。

层级	职责	注意事项
小游戏客户端	展示页面、收集陈词、发起分享、轮询案件状态	不得保存 API Key，不直接写核心集合
云函数 API	鉴权、身份判断、参数校验、Action 分发、错误码	MVP 用一个 api 函数，后期按模块拆分
云数据库	保存用户、案件、陈词、追问、裁决、调用记录	核心集合建议只允许云函数读写
AI 代理函数	限流、拼 Prompt、调用 DeepSeek、解析 JSON	失败时返回可理解错误，不让前端直连 AI

3. 案件生命周期与联机流程

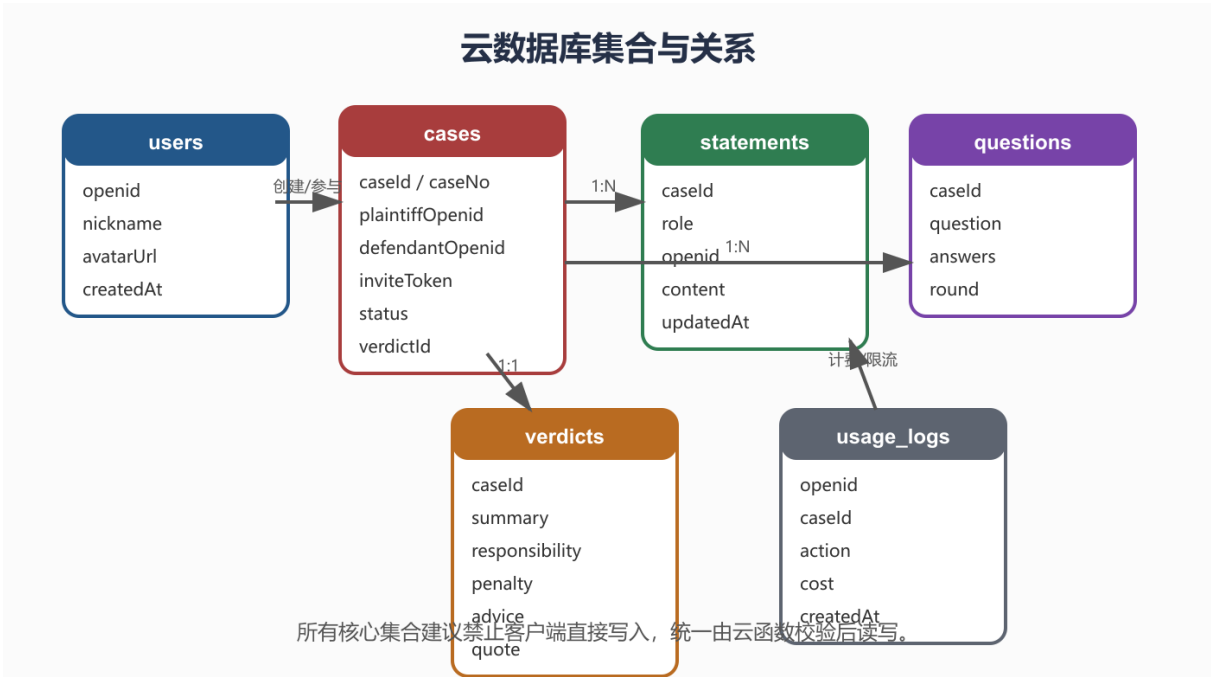


MVP 不需要 WebSocket。用户进入页面、提交陈词、等待对方时调用 getCase；等待状态下每 3-5 秒轮询一次。这样实现简单，足够支撑轻量情侣互动场景。

阶段	客户端动作	云函数动作	状态变化
----	-------	-------	------

创建案件	原告点击“我要起诉”并输入案由	生成 caseld、caseNo、inviteToken, 写入 cases	created
邀请对方	分享带 caseld 和 inviteToken 的链接	无, 等待对方进入	created
被告加入	被告点击链接进入	校验 token, 绑定 defendantOpenid	created
双方陈词	双方分别提交 500 字内陈词	写入 statements, 检查双方是否都提交	both_submitted
AI 追问	点击请 AI 法官追问	读取案情, 生成问题, 写入 questions	questioning
生成裁决	双方回答后点击生成裁决	调用 AI, 写入 verdicts, 更新案件	verdict_done
案卷归档	查看历史案件	按 openid 查询参与案件	archived/展示

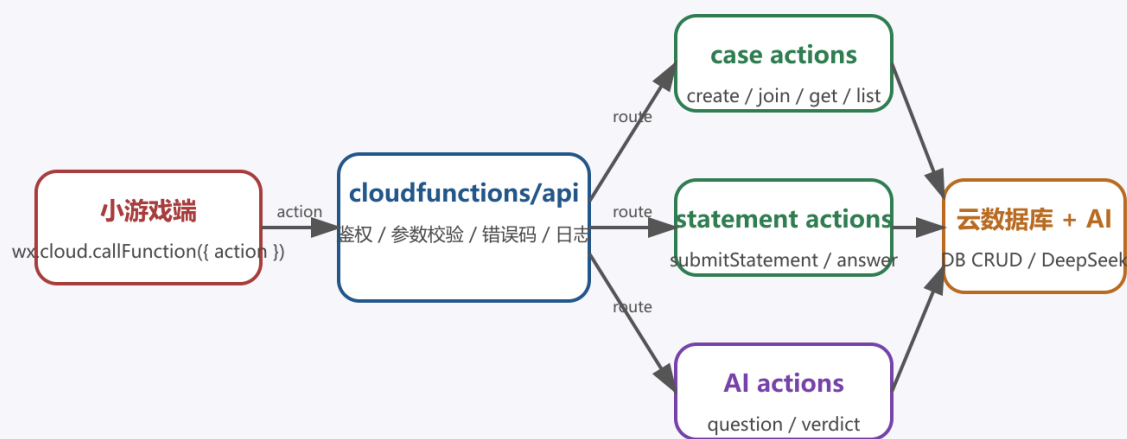
4. 云数据库设计



集合	核心字段	用途
users	openid、nickname、avatarUrl、createdAt、lastLoginAt	保存用户基础信息和登录记录
cases	caseId、caseNo、title、plaintiffOpenid、defendantOpenid、inviteToken、status、verdictId	案件主表，负责身份和状态流转
statements	caseId、role、openid、content、createdAt、updatedAt	保存原告和被告陈词
questions	caseId、question、plaintiffAnswer、defendantAnswer、round、createdAt	保存 AI 追问和双方补充回答
verdicts	caseId、summary、focusPoints、responsibility、reasoning、penalty、settlementAdvice、quote	保存最终判决书结构化结果
usage_logs	openid、caseId、action、cost、createdAt	保存 AI 调用、次数限制和后续商业化依据

5. 云函数 API 分发

统一云函数 API 分发设计



前期一个 api 函数便于迭代；用户量和复杂度上来后再拆 caseApi、aiApi、archiveApi、paymentApi。

MVP 阶段建议只建一个 cloudfunctions/api，通过 action 字段分发。这样迭代快、部署少、调试集中。等功能稳定后，再拆成 caseApi、aiApi、archiveApi、paymentApi。

Action	入参	返回	说明
login	无	openid、用户信息	初始化用户记录
createCase	title	caseId 、 caseNo 、 inviteToken 、 sharePath	原告创建案件
joinCase	caseId 、 inviteToken	role、 case	被告通过链接进入并绑定身份
getCase	caseId	case、 statements、 question、 verdict	页面刷新和轮询使用
submitStatement	caseId、 content	case status	按 openid 自动判断原告/被告
generateQuestion	caseId	question	双方陈词后生成一轮追问
submitAnswer	caseId、 answer	question status	按身份写入对应回答

generateVerdict	caseId	verdict	调用 AI 生成结构化裁决
listCases	status 可选	case list	历史案卷列表

6. 权限、安全与费用控制

- 只有原告和被告能查看案件详情；其他用户即使知道 caseId 也不能读取。
- inviteToken 必须参与 joinCase 校验，不能只靠 caseId 做权限。
- 一个案件只能绑定一个被告，裁决后默认不允许修改陈词。
- 每个用户每天限制 AI 调用次数，例如免费 3-5 次；每个案件最多生成一次正式裁决。
- 陈词限制 500 字，追问回答限制 300 字，防止成本失控和无关内容刷入。
- usage_logs 记录 action 和 cost，为后续广告奖励、次数包、会员权益做准备。

7. AI 调用方案

步骤	处理内容
输入整理	读取案件标题、双方陈词、AI 追问和回答，过滤空值与超长内容
Prompt 约束	明确产品是娱乐互动，不提供法律、医疗、心理诊断或投资建议
结构化输出	要求 AI 返回 JSON：摘要、争议焦点、责任比例、理由、处罚、和解建议、分享语
结果校验	责任比例合计必须为 100，缺字段时补默认值，失败时返回 AI_SERVICE_FAILED
结果落库	写入 verdicts，更新 cases.status = verdict_done，记录 usage_logs

8. MVP 落地路线

- 建立微信小游戏项目骨架，并接入云开发环境。
- 创建数据库集合和基础权限规则，核心写入交给云函数。
- 实现 api 云函数的 createCase、joinCase、getCase。
- 实现双方陈词、状态轮询、等待对方页面。
- 实现 generateVerdict，先跳过多轮追问也可以上线内测。

- 加入判决书分享卡和历史案卷。
- 稳定后再补 AI 追问、次数限制、广告激励或充值次数。

9. 当前最推荐的实现策略

推荐路线

先做微信云开发版，不买服务器。把现有 Web MVP 的核心流程迁成小游戏页面，把 Node 后端能力重写为云函数 action。等用户量、付费和运营需求明确后，再评估是否迁到自有 Node.js 后端 + PostgreSQL/MongoDB + Redis。

附录：错误码建议

错误码	含义
CASE_NOT_FOUND	案件不存在
INVALID_INVITE_TOKEN	邀请码不正确
FORBIDDEN	当前用户无权访问
CASE_ALREADY_VERDICTED	案件已经裁决
STATEMENT_TOO_LONG	陈词超过限制
AI_QUOTA_EXCEEDED	AI 调用次数不足
AI_SERVICE_FAILED	AI 服务调用失败